

Foundations of Cybersecurity

Design and Implementation Report: Timestamping Service (TSS)

Lorenzo Bernardini

Amedeo Meschini

Academic Year 2025–2026

Contents

1	Introduction	3
2	Cryptographic Architecture and Key Separation	3
2.1	Design Rationale for Key Separation	3
2.2	Connection Key Pair ($pubK_c, privK_c$)	4
2.3	Timestamping Key Pair ($pubK_{ts}, privK_{ts}$)	4
3	User Management and Credential Storage	5
3.1	Pre-registration Model and Token Quotas	5
3.2	Plaintext Salts and Offline Attacks	6
3.2.1	Role and Benefits of the Salt	6
3.2.2	The Problem with Plaintext Salts	6
3.2.3	Countermeasures vs. Design Choice	6
4	Shared Module (include/common.hpp)	7
4.1	Canonical Timestamp Payload Construction	7
4.2	Streaming File Hashing and Data Minimization	7
5	Offline Verification Tool (tss_verify)	7
5.1	Architectural Purpose and Threat Model	7
5.2	Token Structure and Signature Format	8
5.3	CLI and Operational Modes	9
5.3.1	Supported Command-Line Parameters	9
5.3.2	Operational Use Cases	9
6	Server Architecture (tss_server)	10
6.1	Server Initialization and Key Management	11
6.2	In-Memory User State and Concurrency Control	11
6.3	Salted Credential Verification and Runtime Accounting	12
6.4	Defenses at Authentication Boundary	12
6.5	TLS 1.3 and Concurrency	13
6.5.1	Enforcing Modern TLS 1.3 Configuration	13
6.5.2	Multi-Threaded Connection Handling and Graceful Shutdown	13
6.6	Session State and Authentication (LOGIN)	13
6.7	Operations: TIMESTAMP and BALANCE	14

6.7.1	Quota Balance Query (BALANCE)	14
6.7.2	Timestamp Token Generation (TIMESTAMP)	15
7	Client Architecture (<code>tss_client</code>)	16
7.1	TLS 1.3 Client Setup and Certificate Pinning	16
7.2	Interactive REPL and Operational Primitives	18
8	Edge Cases and Anti-Replay	18
8.1	Rejection of Unauthenticated Operations	18
8.2	Quota Exhaustion Handling ($n_r = 0$)	18
8.3	Weak Hash and Malformed Input Rejection	18
9	Wire Formats and Message Specifications	19
9.1	Transport-Layer Wire Framing Layout	19
9.2	Authentication Messages (LOGIN)	19
9.2.1	LOGIN Request Packet Layout	19
9.2.2	LOGIN Response Packet Layout (Success vs. Error)	19
9.3	Timestamping Messages (TIMESTAMP)	19
9.3.1	TIMESTAMP Request Packet Layout	19
9.3.2	TIMESTAMP Response Packet Layout	20
9.4	Balance Query Messages (BALANCE)	20
9.5	Session Termination Message (QUIT)	20
10	Validation and Demonstration Scenarios	20
10.1	Scenario 1: Nominal Timestamping and Quota Accounting (Alice)	20
10.2	Scenario 2: Graceful Rejection on Exhausted Quota (Charlie)	20
10.3	Scenario 3: Standalone Verification and Tamper Detection (<code>tss_verify</code>)	21

1 Introduction

A **Time Stamping Authority (TSA)** provides trusted, cryptographically verifiable proof that a specific electronic record existed at a precise point in time without having been altered subsequently.

Our implementation follows the **Data Minimization Principle**: clients never transmit the raw document content over the network. Instead, they submit only a cryptographic hash digest of the document ($\text{hash} = \text{SHA-256}(\text{document})$). The TSA binds this digest to its authoritative system time and generates a digital signature over the bundle:

$$\text{Token} = \text{Sign}_{\text{priv}K_{ts}}(\text{hash} \parallel \text{time}) \quad (1)$$

This design guarantees privacy, minimizes network overhead, and enables offline verification by arbitrary third parties.

2 Cryptographic Architecture and Key Separation

A key security decision in our design is the **strict separation of cryptographic duties**. Rather than reusing a single key pair for multiple operations, the server operates with two independent public/private key pairs:

1. **Connection Key Pair** ($\text{pub}K_c, \text{priv}K_c$): Dedicated exclusively to establishing the secure TLS communication channel.
2. **Timestamping Key Pair** ($\text{pub}K_{ts}, \text{priv}K_{ts}$): Dedicated exclusively to signing timestamp tokens at the application layer.

2.1 Design Rationale for Key Separation

A central design principle is avoiding key reuse across different cryptographic protocols and trust boundaries. Reusing a single key pair for both transport-level TLS encryption and application-level timestamp signing would introduce significant vulnerabilities. We isolate these two functions for three technical reasons:

1. **Impact of Key Compromise and Revocation:**

The connection private key $\text{priv}K_c$ is used in TLS network handshakes. If $\text{priv}K_c$ is compromised, the certificate must be immediately revoked via Certificate Revocation Lists (CRL) or OCSP.

If the same key were used to sign timestamps, an attacker with the stolen key could arbitrarily forge new timestamp tokens with fabricated past dates (backdating). Verifiers would no longer be able to distinguish genuine historical timestamps from forged ones. Revoking the key would effectively destroy the validity of **all past timestamps** ever issued by the service. By keeping $\text{priv}K_{ts}$ strictly separate, a TLS breach or certificate revocation has zero impact on the validity of historical timestamps.

2. **Different Lifespans (Short-Term vs. Long-Term Validity):**

TLS certificates are temporary by design: they expire and are replaced frequently. In contrast, a digital timestamp must remain verifiable for years or decades (e.g., to prove when a legal contract was signed).

If the server used its TLS key to sign timestamps, renewing the TLS certificate would create a serious issue: anyone trying to verify an older timestamp years later would have to rely on an expired, obsolete certificate. Having a separate timestamping key ($\text{pub}K_{ts}$) gives the TSA a stable, long-term identity that does not change every time the web connection certificate is renewed.

3. Separation of Roles and Network Exposure:

The two keys perform entirely different tasks and face different risks. The TLS key is on the "front line": it interacts directly with the public network during every connection attempt, making it naturally more exposed to network-level bugs and scanning.

In contrast, the timestamping key is kept protected inside the application. It never handles network packets directly; it is invoked only after a user has successfully authenticated and their request has been validated. Separating the keys ensures that even if a flaw affects the network/TLS layer, the key used to issue trustworthy timestamps remains safe and isolated.

2.2 Connection Key Pair ($pubK_c, privK_c$)

The connection key pair is based on **Elliptic Curve Cryptography (ECC)**, specifically the **NIST P-256** curve (prime256v1).

Advantages of NIST P-256

- **High Security Margin with Short Keys:** A 256-bit ECC key offers an equivalent security level to a 3072-bit RSA key, significantly reducing memory footprint.
- **Computational Efficiency:** ECC point operations require less processing power and bandwidth than large RSA exponentiations, enabling low-latency TLS handshakes.

```
1 # 1. Connection Key Pair (P-256)
2 openssl ecparam -name prime256v1 -genkey -noout \
3   -out "$CERTS_DIR/server_conn.key"
4 chmod 600 "$CERTS_DIR/server_conn.key"
5
6 openssl req -new -x509 \
7   -key "$CERTS_DIR/server_conn.key" \
8   -out "$CERTS_DIR/server_conn.crt" \
9   -days 365 \
10  -subj "/CN=localhost/O=FoC-TSA/C=IT/ST=Italy/L=University" \
11  -extensions v3_ca \
12  -addext "subjectAltName=IP:127.0.0.1,DNS:localhost"
```

Design Assumptions: Self-Signed Certificate and X.509 Format Because our system is developed and validated in a local development environment running on `localhost` (127.0.0.1), publicly trusted Certificate Authorities (CAs) cannot issue certificates for private loopback addresses. We therefore employ a **self-signed X.509v3 certificate** where the server acts as its own root of trust. Clients are configured to pin and trust this specific `server_conn.crt` certificate.

The X.509 format was selected as it represents the universal standard for public key certificates, guaranteeing interoperability across standard network and TLS libraries. We explicitly include the `subjectAltName` (SAN) extension to bind both the IP address (127.0.0.1) and domain name (`localhost`), fulfilling modern TLS 1.3 hostname validation requirements.

2.3 Timestamping Key Pair ($pubK_{ts}, privK_{ts}$)

For application-level timestamp generation, we selected the **NIST P-384** curve (`secp384r1`) with SHA-384.

```

1 # 2. Timestamping Key Pair (P-384)
2 openssl ecparam -name secp384r1 -genkey -noout \
3   -out "$CERTS_DIR/server_ts.key"
4 chmod 600 "$CERTS_DIR/server_ts.key"
5
6 openssl ec -in "$CERTS_DIR/server_ts.key" -pubout \
7   -out "$CERTS_DIR/server_ts.pub"

```

Why P-384 for Long-Term Non-Repudiation? While connection keys protect session confidentiality during active data transit, digital timestamps must remain valid over extended periods (e.g., 10 to 30 years). The larger key space of P-384 (providing 192 bits of cryptographic strength) ensures long-term resistance against future cryptanalytic advances. The corresponding public key is exported to `server_ts.pub` to allow anyone to verify timestamps offline.

3 User Management and Credential Storage

3.1 Pre-registration Model and Token Quotas

As per the project specifications, user registration is an administrative pre-condition. The server maintains a persistent user registry (`users.json`) where each registered user is provisioned with:

- A distinct `username` and salted password hash.
- A pre-allocated timestamp budget tracked by two counters:
 - n_c : Number of timestamps consumed (initialized to 0).
 - n_r : Number of timestamps remaining (available allowance).

```

1 # User record generation in keygen.sh
2 gen_user() {
3   user="$1"; pass="$2"; nc="$3"; nr="$4"
4   salt=$(openssl rand -hex 16)
5   hash=$(printf '%s%s' "$pass" "$salt" | openssl dgst -sha256 -hex | awk '{print $2}')
6   cat << EOF
7   "$user": {
8     "salt": "$salt",
9     "password_hash": "$hash",
10    "nc": $nc,
11    "nr": $nr
12  }
13 EOF
14 }

```

The generated `users.json` establishes demo user profiles tailored for testing:

- **alice** ($n_c = 0, n_r = 100$): Standard user profile for nominal timestamping operations.
- **bob** ($n_c = 0, n_r = 50$): Secondary profile to validate concurrent client access.
- **charlie** ($n_c = 10, n_r = 0$): Depleted account profile to test graceful rejection when balance is exhausted.

3.2 Plaintext Salts and Offline Attacks

3.2.1 Role and Benefits of the Salt

To ensure user passwords are never stored in plaintext, the generation script computes:

$$\text{password_hash} = \text{SHA-256}(\text{password} \parallel \text{salt}) \quad (2)$$

where `salt` is a 16-byte (128-bit) random hexadecimal string generated via `openssl rand`.

The unique salt achieves two security objectives:

1. **Neutralization of Precomputed Tables (Rainbow Tables):** An attacker cannot use precomputed hash tables or global dictionary rainbow tables across the database. Since every user has a distinct 128-bit salt, generating precomputed tables would require 2^{128} separate tables.
2. **Elimination of Cross-User Hash Matching:** If two users choose identical passwords, their stored hashes will differ completely due to distinct salts.

3.2.2 The Problem with Plaintext Salts

While a salt prevents the use of precomputed tables, storing it in plaintext does not provide complete protection against offline attacks. If an attacker compromises the server’s database (`users.json`), the plaintext salt does not prevent a targeted dictionary attack against an individual user.

Because both the salt s and the digest h are accessible in the database, an attacker can iterate through a dictionary of candidate passwords $\mathcal{D} = \{w_1, w_2, \dots, w_m\}$ and compute $h' = \text{SHA-256}(w_i \parallel s)$ locally. Modern GPUs can evaluate billions of SHA-256 hashes per second, making short or common passwords vulnerable to rapid discovery.

3.2.3 Countermeasures vs. Design Choice

In high-security enterprise environments, this vulnerability is mitigated through two primary techniques:

1. **Secret Salt (Pepper):** An additional application-level secret key K_{pep} is stored outside the database (e.g., inside a Hardware Security Module) and included during hashing: $H(\text{password} \parallel \text{salt} \parallel K_{\text{pep}})$. A leak of the database alone does not reveal K_{pep} .
2. **Memory-Hard Key Derivation Functions (KDFs):** Employing algorithms such as **Argon2id**, **bcrypt**, or **PBKDF2** configured with high iteration counts and memory cost. This intentionally increases the computational cost of each trial, rendering dictionary enumeration impractical.

Rationale for Our Implementation Choice We store the salt in plaintext within `users.json` because, by cryptographic standard (e.g., RFC 2898), a salt is public metadata meant to randomize hashes, not a secret key. Encrypting the salt locally would introduce a circular key-storage problem. While enterprise environments deploy dedicated infrastructure like HSMs for secret *peppers* or memory-heavy KDFs, our choice reflects the educational nature of this project: prioritizing a lightweight, self-contained prototype modeled after standard Unix credential stores (such as `/etc/shadow`). Credential confidentiality in transit is guaranteed by the established TLS 1.3 encrypted tunnel.

4 Shared Module (include/common.hpp)

To avoid code duplication and maintain a unified cryptographic standard across the client, server, and offline verifier, we implemented a shared header library (include/common.hpp).

4.1 Canonical Timestamp Payload Construction

Cryptographic signatures require strict determinism: the binary representation of the signed payload must be identical across heterogeneous system architectures (e.g., Little-Endian x86 vs. Big-Endian ARM).

The service binds the document hash to the Unix epoch timestamp by generating a fixed **40-byte binary payload**:

$$\text{Payload}_{40\text{B}} = \text{Hash}_{32\text{B}} \parallel \text{encode_be64}(\text{Time}_{8\text{B}}) \quad (3)$$

```
1 inline std::vector<uint8_t> build_timestamp_payload(const std::string& hash_hex,
2                                                     uint64_t timestamp) {
3     auto hash_bytes = hex_to_bytes(hash_hex);
4     if (hash_bytes.size() != 32) {
5         throw std::invalid_argument("SHA-256 hash must be exactly 32 bytes");
6     }
7
8     // Convert epoch time to Big-Endian (Network Byte Order)
9     uint64_t be_time = htobe64(timestamp);
10
11     std::vector<uint8_t> payload;
12     payload.reserve(32 + sizeof(uint64_t));
13     payload.insert(payload.end(), hash_bytes.begin(), hash_bytes.end());
14
15     const uint8_t* time_bytes = reinterpret_cast<const uint8_t*>(&be_time);
16     payload.insert(payload.end(), time_bytes, time_bytes + sizeof(uint64_t));
17
18     return payload;
19 }
```

4.2 Streaming File Hashing and Data Minimization

To enforce the Data Minimization Principle without loading large files entirely into RAM, common.hpp implements chunked streaming file hashing using OpenSSL's `EVP_DigestUpdate` with a 16 KB buffer:

$$H(x) = \text{SHA-256}(b_1 \parallel b_2 \parallel \dots \parallel b_n) \quad (4)$$

This allows client applications to hash multi-gigabyte documents locally with constant memory consumption before transmitting the resulting 32-byte digest over the network.

5 Offline Verification Tool (tss_verify)

5.1 Architectural Purpose and Threat Model

Timestamp tokens must be verifiable by arbitrary third parties without requiring an active network connection or service credentials.

The standalone CLI executable `tss_verify` fulfills the mandatory project deliverable by verifying:

1. **Token Integrity and Authenticity:** Validates that the token bundle $\langle \text{hash}, \text{time}, \text{signature} \rangle$ was produced by the TSA key $\text{priv}K_{ts}$ and has not suffered tampering.
2. **Document Binding:** (Optional) Computes the local SHA-256 digest of a specified file on disk and confirms that it matches the hash recorded in the token.

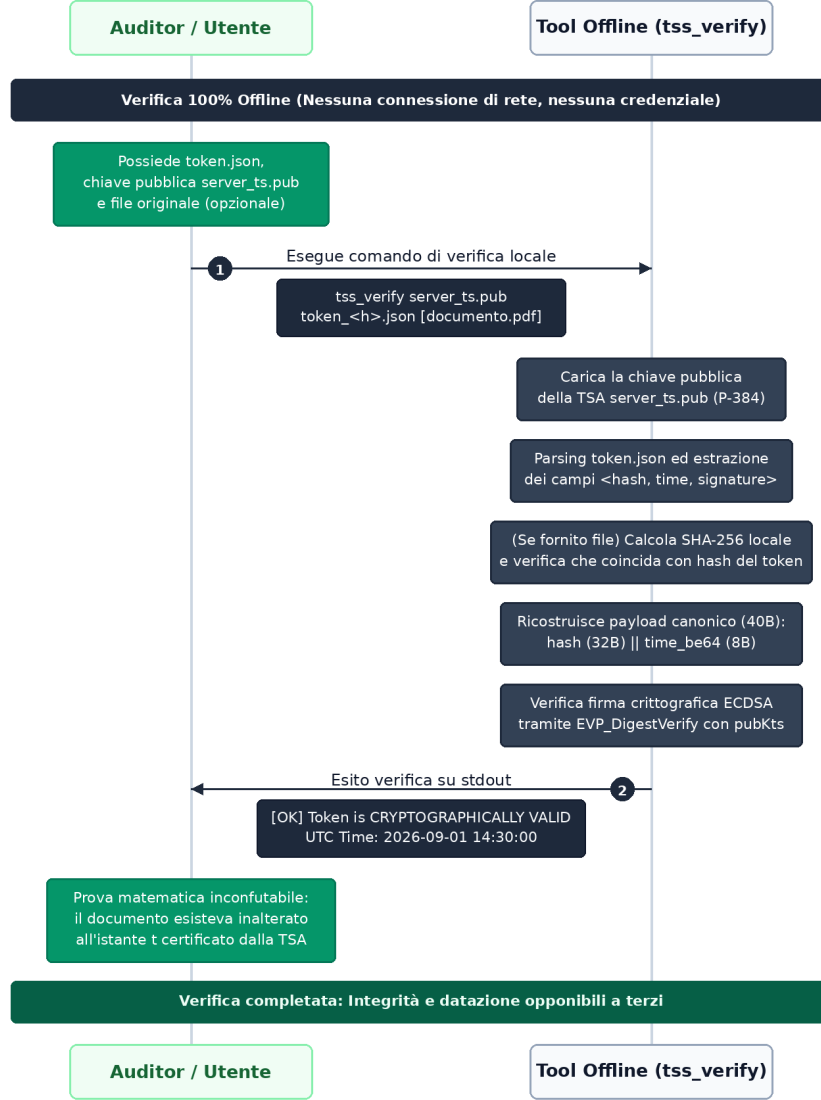


Figure 1: Universal offline verification workflow using `tss_verify` and the long-term TSA public key ($\text{pub}K_{ts}$).

As illustrated in Figure 1, the verifier operates strictly locally. It extracts the components from the token JSON, reconstructs the deterministic 40-byte payload, and uses OpenSSL's `EVP_DigestVerifyInit` and `EVP_DigestVerifyFinal` to validate the ECDSA signature against `server_ts.pub`.

5.2 Token Structure and Signature Format

The token is exchanged and stored in JSON format as follows:

```

1 {
2   "hash": "d4bbbf8c4b7db800e39efff1f8c94a17b68b83600e092441876a88c72d0cd222",
3   "time": 1787926643,
4   "signature": "3066023100f39f755f46679e...023100f37144d0..."
5 }

```

The `signature` field contains 208 hexadecimal characters, corresponding to exactly **104 binary bytes**. This structure adheres to the **ASN.1 / DER** standard (ECDSA-Sig-Value):

- **DER Sequence Header (2 bytes):** Tag 0x30 (Sequence) followed by Length 0x66 (102 bytes).
- **Component r (51 bytes):** Tag 0x02 (Integer), Length 0x31 (49 bytes), optional zero padding byte 0x00 (ensuring positive integer representation), followed by the 48-byte big-endian encoding of r .
- **Component s (51 bytes):** Tag 0x02 (Integer), Length 0x31 (49 bytes), optional zero padding byte 0x00, followed by the 48-byte big-endian encoding of s .

Total binary size: $2 + 51 + 51 = 104$ bytes, which yields $104 \times 2 = 208$ hexadecimal characters.

5.3 CLI and Operational Modes

The `tss_verify` binary is engineered to support multiple operational workflows, catering to casual end-users, automated CI/CD pipelines, and forensic auditors.

5.3.1 Supported Command-Line Parameters

The tool accepts the following flags:

Option Flag	Argument Type	Description and Default Behavior
<code>--token <file.json></code>	Filepath	Path to the JSON token bundle containing <code>hash</code> , <code>time</code> , and <code>signature</code> .
<code>--file <doc></code>	Filepath (Optional)	Path to the physical document on disk to verify Document Binding via streaming SHA-256.
<code>--pubkts <key.pub></code>	Filepath (Optional)	Path to the TSA public key in PEM format (defaults to <code>certs/server.ts.pub</code>).
<code>--hash <hex></code>	64-char Hex String	Manual SHA-256 digest (used in headless/scripting mode without token file).
<code>--time <epoch></code>	64-bit Unsigned Int	Manual Unix epoch timestamp (used in headless/scripting mode).
<code>--sig <hex></code>	Hex String (DER)	Manual ECDSA signature string (used in headless/scripting mode).
<code>--help, -h</code>	None	Displays command usage instructions and exit code specifications.

Table 1: `tss_verify` command-line options and parameters.

5.3.2 Operational Use Cases

Depending on the arguments supplied, the verification engine operates in one of four distinct modes:

1. Autonomous Token Verification (Integrity and Authenticity Only) When auditing a token independently without requiring access to the original underlying file:

```
./build/tss_verify --token token_d4bbbf8c4b7db800.json
```

The tool parses the JSON bundle, reconstructs the 40-byte canonical binary payload, and asserts that the recorded hash and timestamp were signed by the TSA private key ($privK_{ts}$).

2. Full Document Binding Verification (Legal Non-Repudiation) When verifying that a specific file on disk is strictly bound to the issued timestamp certificate:

```
./build/tss_verify --token token_d4bbbf8c4b7db800.json --file mio_documento.txt
```

In addition to validating the digital signature, the tool streams `mio_documento.txt` through `sha256_file()` and checks bitwise identity against `token.hash_hex`. Any divergence immediately halts the routine before evaluating the signature, thwarting document substitution attacks.

3. Custom Authority Key Audit When an external relying party stores the TSA's public certificate in an institutional trust store:

```
./build/tss_verify --token token.json --pubkts /path/to/trusted_tsa.pub
```

This overrides the compile-time default path, allowing cross-verification against arbitrary authority keys.

4. Headless Scripting / In-Memory Piping Mode To facilitate automated testing and integration into higher-level services without writing temporary JSON files to disk, the verifier can be driven entirely via raw command-line parameters:

```
./build/tss_verify --hash d4bbbf8c... --time 1788860433 --sig 30650231...
```

Exit Codes for Scripting Automation To enable deterministic error handling in automated shell scripts, `tss_verify` returns POSIX exit codes:

- **0 (Success):** Cryptographic signature is valid and document binding (if requested) is confirmed.
- **1 (Operational Error):** File I/O failure, malformed JSON structure, or missing required parameters.
- **2 (Verification Failure):** Tampered token, invalid cryptographic signature, or document hash mismatch.

6 Server Architecture (`tss_server`)

The timestamping server (`tss_server`) is the central authoritative component of the architecture, responsible for authenticating clients, enforcing quota limits, and issuing signed timestamp tokens.

6.1 Server Initialization and Key Management

Upon startup, the server performs a deterministic sequence of initialization routines to guarantee environment integrity before opening network sockets:

1. **Configuration and Parameter Parsing:** Evaluates command-line flags (listening port, certificate paths, database file) and assigns standardized default fallbacks configured in `common.hpp`.
2. **TLS Credential Validation:** Confirms the presence and accessibility of the connection certificate (`server_conn.crt`) and private key (`server_conn.key`) required for transport-layer security.
3. **Timestamping Private Key Ingestion:** Ingests the authoritative P-384 private key (`server_ts.key`) into an OpenSSL `EVP_PKEY` structure using high-level PEM decoders (`PEM_read_PrivateKey`).

```
1 // Loading long-term TSA signing private key (privKts)
2 EVP_PKEY* load_private_key(const std::string& key_path) {
3     FILE* kf = fopen(key_path.c_str(), "r");
4     if (!kf) return nullptr;
5
6     EVP_PKEY* pkey = PEM_read_PrivateKey(kf, nullptr, nullptr, nullptr);
7     fclose(kf);
8     return pkey;
9 }
```

6.2 In-Memory User State and Concurrency Control

To minimize latency and reduce disk I/O during runtime transactions, the server deserializes `users.json` into an in-memory hash map at boot:

```
std::unordered_map<std::string, UserRecord> g_users;
```

Each `UserRecord` contains the user's 128-bit salt, the precomputed salted hash, and the operational quotas:

- n_c : Number of timestamps consumed by the user.
- n_r : Number of remaining timestamps available in the user's budget.

Thread-Safety and Race Condition Prevention Because the server is designed to service multiple concurrent client connections simultaneously across independent worker threads, all operations accessing or modifying `g_users` are synchronized via a global mutex:

```
std::mutex g_users_mtx;
```

Thread safety is enforced through the RAII pattern using `std::lock_guard<std::mutex>`. This guarantees mutual exclusion during credential checks, balance queries, quota updates, and atomic persistence to disk.

6.3 Salted Credential Verification and Runtime Accounting

During client authentication, the server executes runtime verification against the in-memory registry:

$$H_{\text{computed}} = \text{SHA-256}(\text{password}_{\text{client}} \parallel \text{salt}_{\text{record}}) \quad (5)$$

The computed digest is compared against the stored `password_hash`. If valid, the session is authorized and granted access to subsequent timestamping and balance operations.

```
1 bool verify_user_credentials(const std::string& username,
2                             const std::string& password) {
3     std::lock_guard<std::mutex> lock(g_users_mtx);
4
5     auto it = g_users.find(username);
6     if (it == g_users.end()) {
7         return false; // User not found
8     }
9
10    const UserRecord& rec = it->second;
11    std::string computed_hash = tss::sha256_string(password + rec.salt);
12    return (computed_hash == rec.password_hash);
13 }
```

6.4 Defenses at Authentication Boundary

The authentication process incorporates several security countermeasures:

1. **Memory Safety and Input Type Validation:**

The application avoids raw C-style string buffers and fixed-size arrays susceptible to buffer overflow vulnerabilities. All user inputs are parsed into dynamically managed C++ Standard Library structures (`std::string`). Requests are deserialized using guarded `try-catch` blocks that enforce expected JSON types, safely rejecting malformed payloads without crashing the server thread.

2. **Anti-DoS Resource Bounding:**

To prevent resource exhaustion attacks where malicious clients send excessively large frames to allocate disproportionate heap memory, the network framing enforces a strict compile-time threshold (`MAX_PAYLOAD_SIZE = 64 KB`). Payloads exceeding this threshold are dropped immediately at the wire level before memory allocation.

3. **Multi-Layer Replay Protection and Application-Level Session Binding:**

Adversaries attempting to record and retransmit valid authentication or operational packets are thwarted across two distinct layers:

- *Transport Layer (TLS 1.3):* Uses ephemeral keys (PFS) and authenticated encryption (AEAD) with strict record sequence numbering, making recorded network ciphertexts non-replayable across sessions.
- *Application Layer:* During the login handshake, the server responds with an authoritative session nonce (N_s). To render the application protocol secure even in the presence of TLS-terminating reverse proxies, all subsequent requests (`TIMESTAMP`, `BALANCE`, `QUIT`) are explicitly bound to this session nonce N_s and a strictly increasing sequence counter (`seq`). All server responses mirror N_s and `seq`, enabling mutual verification and preventing in-session replay, cross-session injection, and reflection attacks.

6.5 TLS 1.3 and Concurrency

All client-server interactions are encapsulated within a strictly enforced TLS 1.3 channel over POSIX stream sockets.

6.5.1 Enforcing Modern TLS 1.3 Configuration

To eliminate legacy cryptographic vulnerabilities, the server explicitly mandates TLS 1.3 via OpenSSL context primitives:

```
1 SSL_CTX* ctx = SSL_CTX_new(TLS_server_method());
2 // Mandate TLS 1.3 as the strict minimum protocol version
3 SSL_CTX_set_min_proto_version(ctx, TLS1_3_VERSION);
4 SSL_CTX_use_certificate_file(ctx, cert_file.c_str(), SSL_FILETYPE_PEM);
5 SSL_CTX_use_PrivateKey_file(ctx, key_file.c_str(), SSL_FILETYPE_PEM);
6 SSL_CTX_check_private_key(ctx);
```

This guarantees that every established session negotiates Perfect Forward Secrecy (PFS) and Authenticated Encryption with Associated Data (AEAD).

6.5.2 Multi-Threaded Connection Handling and Graceful Shutdown

The networking subsystem uses a master-worker concurrency model:

1. The main thread listens on the configured TCP port (8443) and executes the `accept()` loop.
2. Upon accepting an incoming client connection, the server instantiates an independent SSL session structure and dispatches a dedicated worker thread (`std::thread`) to manage the connection life cycle.
3. **Graceful Shutdown:** POSIX signal handlers for `SIGINT` and `SIGTERM` toggle an atomic execution flag (`std::atomic<bool> g_running`) and shut down the master socket, releasing system descriptors and deallocating OpenSSL structures cleanly.

6.6 Session State and Authentication (LOGIN)

Each client connection is governed by an isolated session state machine instantiated within the worker thread:

1. **Initial Unauthenticated State:** Upon completing the TLS handshake, the connection defaults to an unauthenticated state. Any non-login command received is immediately rejected with an "Access denied" error frame.
2. **The LOGIN Handshake:** The client initiates authentication by sending:

```
{"cmd": "LOGIN", "username": "u", "password": "p"}
```

The server validates the credentials against the salted in-memory user registry. Upon success:

- Generates a fresh 16-byte cryptographically secure server nonce $N_s \leftarrow \text{CSPRNG}(16)$.
- Sets the session state to *Authenticated* bound to username u .
- Initializes the application-level sequence counter `expected_seq` $\leftarrow 1$.

- Responds with `{"status": "OK", "nonce_s": "Ns"}`.

If authentication fails, the server returns an explicit error message and closes the TCP connection to mitigate brute-force and credential-stuffing attempts.

3. **Enforcing Anti-Replay Monotonicity and Session Binding (seq, N_s):** Every command following a successful login must include both the authoritative session nonce N_s and a strictly monotonic 64-bit integer seq. The server validates that $\text{nonce}_s == N_s$ and that $\text{seq} == \text{expected_seq}$, subsequently incrementing the expected sequence number ($\text{expected_seq} \leftarrow \text{expected_seq} + 1$). Any mismatch in either field triggers an immediate session abort.

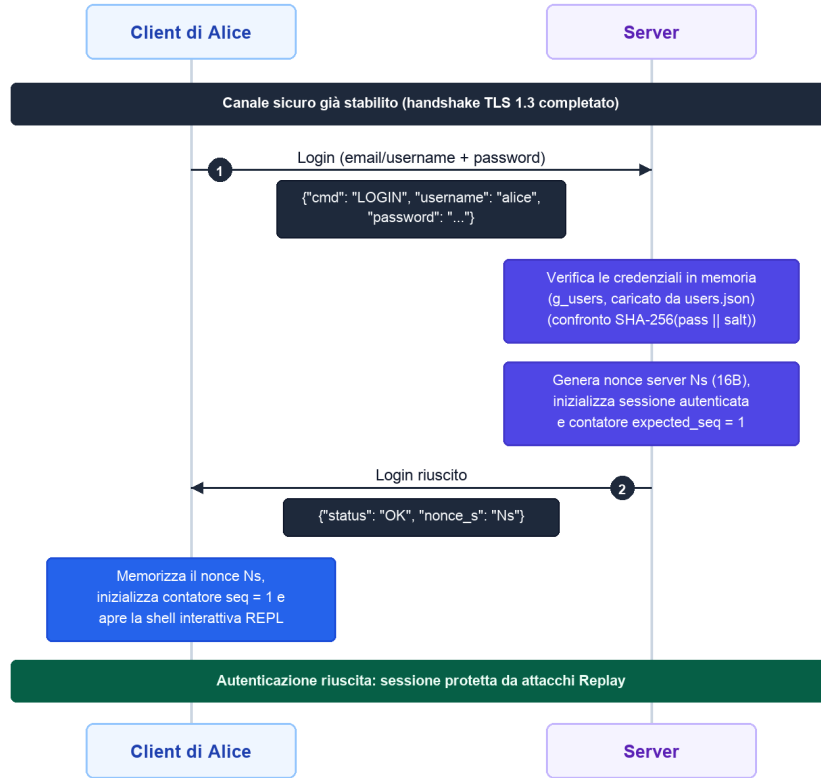


Figure 2: Session authentication and cryptographic nonce exchange protocol (LOGIN).

The interaction flow is shown in Figure 2, highlighting the generation of nonces on both endpoints to bind session freshness.

6.7 Operations: TIMESTAMP and BALANCE

Once authenticated, clients interact with the service through two core operational primitives:

6.7.1 Quota Balance Query (BALANCE)

The client queries its current timestamping budget by transmitting:

```
{"cmd": "BALANCE", "nonce_s": "Ns", "seq": seq}
```

The server verifies both the session nonce N_s and the sequence counter, retrieves the user's record under mutual exclusion (`std::mutex`), and returns:

```
{"status": "OK", "nc": nc, "nr": nr, "nonce_s": "Ns", "seq": seq}
```

6.7.2 Timestamp Token Generation (TIMESTAMP)

The core service operation binds a document hash to the authoritative system clock and signs the bundle:

1. **Request Formulation:** The client transmits the 32-byte (64 hex characters) SHA-256 digest of the target document bound to the session:

```
{ "cmd": "TIMESTAMP", "hash": "h", "nonce_s": "N_s", "seq": seq }
```

2. **Syntactic, Session, and Quota Verification:** The server verifies that `nonce_s == N_s`, that `seq == expected_seq`, and that `h` is a valid 64-character hexadecimal string. Under mutex synchronization, it checks the user's available allowance:
 - If $n_r == 0$, the request is gracefully rejected with `{ "status": "ERROR", "message": "Timestamp balance exhausted", "nonce_s": "N_s", "seq": seq }`, leaving server state unaltered.
 - If $n_r > 0$, the server atomically decrements the allowance ($n_r \leftarrow n_r - 1$), increments consumed tokens ($n_c \leftarrow n_c + 1$), and immediately persists the updated database to `users.json`.
3. **Payload Construction and Digital Signing:** The server samples the current Unix epoch time t and constructs the deterministic 40-byte binary payload (`Payload40B = Hash32B || encode_be64( $t_{8B}$ )`).

Following the **Hash-then-Sign** paradigm, ECDSA requires the input data to be compressed to a fixed-length digest matching the elliptic curve size prior to signing. Therefore, the TSA computes the digital signature (ECDSA over NIST P-384) by first hashing the 40-byte bundle with SHA-384:

$$\sigma = \text{Sign}_{\text{priv}K_{ts}}(\text{Payload}) = \text{ECDSA}_{\text{priv}K_{ts}}\left(\text{SHA-384}(\text{Hash}_{32B} \parallel \text{encode_be64}(t_{8B}))\right) \quad (6)$$

In our C++ implementation, this process is handled through OpenSSL's high-level `EVP_DigestSign` API:

```
1 // High-level EVP signing with P-384 and SHA-384
2 std::string sign_timestamp_payload(EVP_PKEY* pkey,
3                                   const std::vector<uint8_t>& payload) {
4     EVP_MD_CTX* ctx = EVP_MD_CTX_new();
5     EVP_DigestSignInit(ctx, nullptr, EVP_sha384(), nullptr, pkey);
6     EVP_DigestSignUpdate(ctx, payload.data(), payload.size());
7
8     size_t sig_len = 0;
9     EVP_DigestSignFinal(ctx, nullptr, &sig_len);
10    std::vector<uint8_t> sig(sig_len);
11    EVP_DigestSignFinal(ctx, sig.data(), &sig_len);
12
13    EVP_MD_CTX_free(ctx);
14    sig.resize(sig_len);
15    return tss::bytes_to_hex(sig);
16 }
```

The server concludes by returning the signed timestamp token, binding it to the session and request sequence:

```
{
  "status": "OK",
  "hash": "h",
  "time": t,
  "signature": " $\sigma_{\text{hex}}$ ",
  "nc":  $n_c$ ,
  "nr":  $n_r$ ,
  "nonce_s": " $N_s$ ",
  "seq": seq
}
```

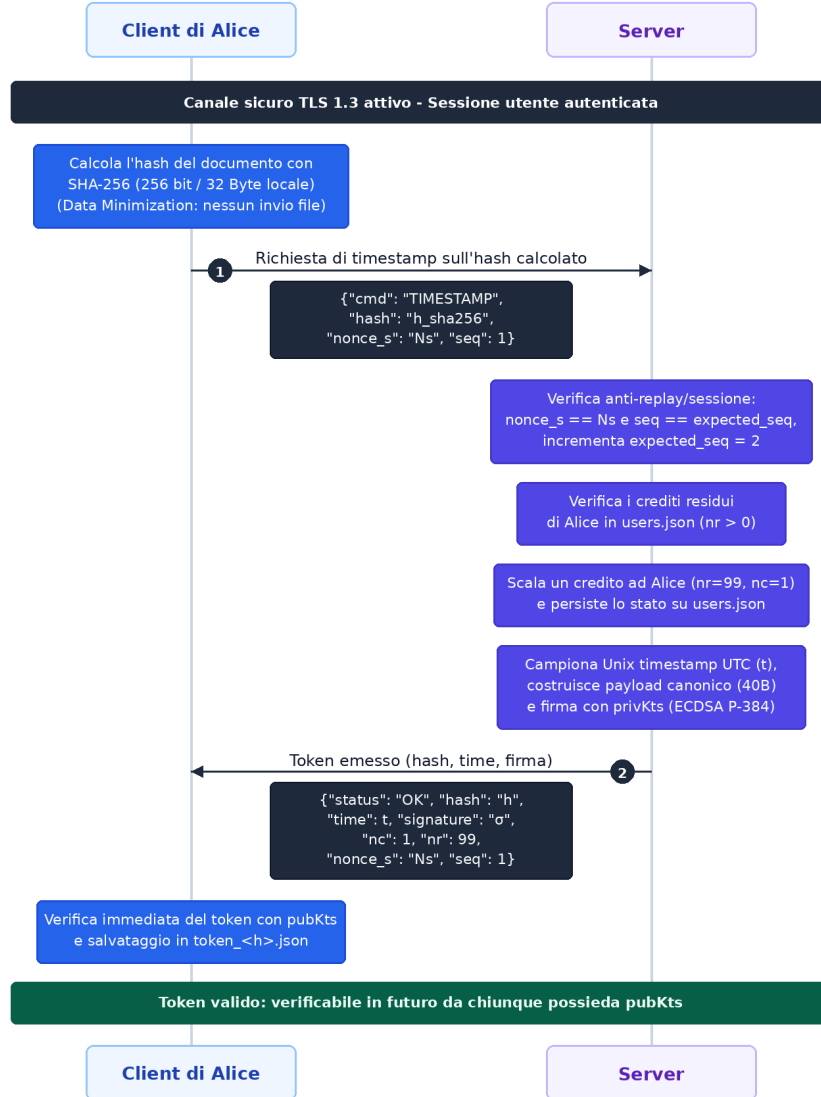


Figure 3: Timestamp request, anti-replay validation, quota accounting, and cryptographic token issuance (TIMESTAMP).

The nominal timestamping sequence is detailed in Figure 3.

7 Client Architecture (tss_client)

The client application (`tss_client`) acts as the end-user interface to the Timestamping Service, enabling secure authentication, token acquisition, balance queries, and offline verification of received tokens.

7.1 TLS 1.3 Client Setup and Certificate Pinning

To thwart Man-in-the-Middle (MitM) attacks on untrusted networks, the client implements **certificate pinning**:

1. **Version Enforcement:** Configures an OpenSSL client context strictly mandating TLS 1.3 via `SSL_CTX_set_min_proto_version(ctx, TLS1_3_VERSION)`.
2. **Trust Anchor Pinning:** Ingests the server's public certificate (`server_conn.crt`) as the trusted authority via `SSL_CTX_load_verify_locations`, enforcing strict peer validation (`SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, ...)`).
3. **Server Name Indication (SNI):** Transmits the requested server hostname via `SSL_set_tlsext_host_name`.
4. **Peer Verification Check:** After completing `SSL_connect`, verifies that `SSL_get_verify_result` returns `X509_V_OK`, confirming the server's cryptographic identity before transmitting any sensitive data.

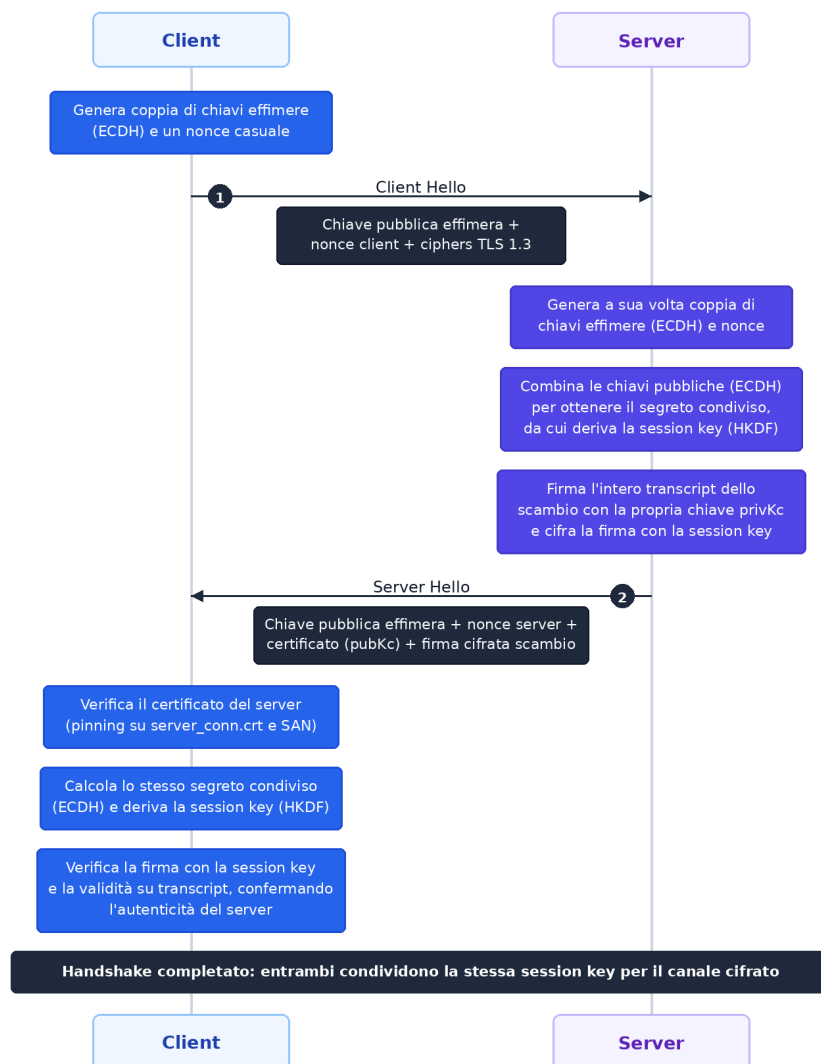


Figure 4: TLS 1.3 handshake sequence and strict certificate pinning verification with $pubK_c$.

Figure 4 illustrates the complete TLS 1.3 handshake, ephemeral ECDH key derivation, transcript signature verification, and trust establishment.

7.2 Interactive REPL and Operational Primitives

The client exposes a terminal-based Read-Eval-Print Loop (REPL) with the following commands:

- **timestamp <file | hex>**: Supports either a local file path or a 64-character SHA-256 hexadecimal string. When a file path is provided, the client computes the hash locally using chunked streaming buffers (`tss::sha256_file`), adhering to the **Data Minimization Principle**. Upon receiving a valid token, the client formats the UTC timestamp and automatically persists the bundle to `token-<hash_prefix>.json`.
- **balance**: Sends an authenticated balance inquiry and prints the user's consumed (n_c) and remaining (n_r) timestamp quotas.
- **verify <token.json>**: Validates a saved token file offline against the server's public key (`server_ts.pub`) using ECDSA/SHA-384 without closing the client session.
- **quit / exit**: Emits a QUIT frame to the server and gracefully shuts down the TLS session and socket descriptors.

8 Edge Cases and Anti-Replay

The system handles several key exceptional paths to maintain deterministic and secure operation:

8.1 Rejection of Unauthenticated Operations

Any command submitted prior to a successful LOGIN handshake is rejected with:

```
{"status": "ERROR", "message": "Access denied: Please login first"}
```

This ensures the server state machine cannot be bypassed.

8.2 Quota Exhaustion Handling ($n_r = 0$)

When an authenticated user whose quota is exhausted (such as demo user `charlie`) issues a `TIMESTAMP` command, the server executes a synchronized check on `g_users`. Detecting $n_r == 0$, the server:

1. Leaves n_c and n_r unchanged in memory.
2. Emits no digital signature and performs no disk write.
3. Transmits `{"status": "ERROR", "message": "Timestamp balance exhausted"}`.

8.3 Weak Hash and Malformed Input Rejection

A timestamping service must enforce strict collision resistance to guarantee the non-repudiation of certified documents. Accepting weak hash algorithms (such as MD5 with 128-bit digests) would introduce severe vulnerabilities, as collision-generation attacks could allow an adversary to construct two distinct documents yielding identical hashes.

To protect the service against collision attacks and corrupted payloads, the server validates all incoming hash digests at the wire framing layer prior to cryptographic processing:

1. Asserts that the submitted hash string contains exactly 64 hexadecimal characters (`doc_hash.length() == 64`), strictly enforcing SHA-256 (32 bytes).

2. Confirms that all characters are valid hexadecimal digits ($[0 - 9, a - f, A - F]$).
3. Rejects non-conforming or truncated inputs immediately with:

```
{
  "status": "ERROR",
  "message": "Formato hash non valido (deve essere una stringa esadecimale SHA-256 di 64 caratteri)"
}
```

4. Preserves the user's operational quota (n_r) without decrementing credits or invoking cryptographic signing routines.

9 Wire Formats and Message Specifications

9.1 Transport-Layer Wire Framing Layout

Length Prefix	JSON Payload
4 B (uint32_t, Big-Endian)	N B (variable, $1 \leq N \leq 64$ KB)
<i>framing header</i>	<i>authenticated + encrypted (TLS 1.3 AEAD)</i>

Table 2: Wire framing layout for all application packets over TLS 1.3.

The 4-byte length prefix ensures deterministic stream framing and enables immediate rejection of frames exceeding `MAX_PAYLOAD_SIZE` (64 KB) before allocating heap buffers.

9.2 Authentication Messages (LOGIN)

9.2.1 LOGIN Request Packet Layout

Command (cmd)	Username (username)	Password (password)
5 B (String: "LOGIN")	1–32 B (String)	Variable (String)

Table 3: Field breakdown and byte dimensions for LOGIN request.

9.2.2 LOGIN Response Packet Layout (Success vs. Error)

Outcome	Status (status)	Payload Field
Success (OK)	2 B (String: "OK")	Server Nonce (nonce_s): 32 B Hex (16 B raw CSPRNG)
Failure (ERROR)	5 B (String: "ERROR")	Diagnostic Message (message): Variable String

Table 4: Field breakdown for successful and failed LOGIN responses.

9.3 Timestamping Messages (TIMESTAMP)

9.3.1 TIMESTAMP Request Packet Layout

Command (cmd)	Digest (hash)	Session Nonce (nonce_s)	Sequence (seq)
9 B ("TIMESTAMP")	64 B Hex (32 B SHA-256)	32 B Hex (16 B CSPRNG)	8 B (uint64_t)

Table 5: Field breakdown and byte dimensions for TIMESTAMP request.

9.3.2 TIMESTAMP Response Packet Layout

Status (status)	Digest (hash)	Time (time)	Signature (signature)	Session (nonce_s)	Seq (seq)
2 B ("OK")	64 B Hex (32 B SHA-256)	8 B (uint64_t Epoch)	208 B Hex (104 B ASN.1 DER)	32 B Hex (16 B CSPRNG)	8 B (uint64_t)

Table 6: Field breakdown and byte dimensions for **TIMESTAMP** response (Token Bundle and Session Reflection).

9.4 Balance Query Messages (BALANCE)

Packet	Field 1	Field 2	Field 3 / 4
Request	Command (cmd): 7 B ("BALANCE")	Session (nonce_s): 32 B Hex	Sequence (seq): 8 B (uint64_t)
Response (OK)	Status (status): 2 B ("OK")	Consumed (nc): 8 B Remaining (nr): 8 B	Session (nonce_s): 32 B Hex Seq (seq): 8 B

Table 7: Field breakdown for **BALANCE** request and response.

9.5 Session Termination Message (QUIT)

Command (cmd)	Session Nonce (nonce_s)	Sequence Number (seq)
4 B (String: "QUIT")	32 B Hex (16 B raw CSPRNG)	8 B (uint64_t Integer)

Table 8: Field breakdown for **QUIT** frame.

10 Validation and Demonstration Scenarios

To validate all requirements established in the project specifications, we developed automated build and verification scripts (`build.sh`, `keygen.sh`). We systematically exercised the system through the three mandatory evaluation scenarios:

10.1 Scenario 1: Nominal Timestamping and Quota Accounting (Alice)

1. **Setup:** User `alice` connects over TLS 1.3, pins `server_conn.crt`, and logs in.
2. **Initial Quota:** Balance query returns $n_c = 0, n_r = 100$.
3. **Hashing & Request:** Client hashes a local sample file (`mio_documento.txt`) via streaming SHA-256 and submits the digest with `seq = 1`.
4. **Token Issuance:** Server signs payload ($h \parallel t$) with $privK_{ts}$ (ECDSA P-384) and returns the token.
5. **Accounting Update:** Balance query returns $n_c = 1, n_r = 99$. The updated state is verified in `users.json`.
6. **Offline Verification:** The client saves `token_*.json` and validates it offline via `tss_verify server_ts.pub token.json mio_documento.txt`, producing a confirmed valid cryptographic verdict.

10.2 Scenario 2: Graceful Rejection on Exhausted Quota (Charlie)

1. **Setup:** User `charlie` ($n_c = 10, n_r = 0$) authenticates over TLS 1.3.
2. **Request:** Charlie submits a valid document hash.

3. **Rejection:** Server evaluates $n_r == 0$ under mutex synchronization and returns an explicit error frame: "Timestamp balance exhausted".
4. **State Integrity:** Server issues no token, creates no signature, and leaves $n_c = 10, n_r = 0$ unaltered on disk.

10.3 Scenario 3: Standalone Verification and Tamper Detection (`tss_verify`)

1. **Nominal Validation:** `tss_verify` is executed with `server_ts.pub`, verifying mathematical signature validity and confirming document binding.
2. **Tampering Experiment A (Modified Document):** When verified against a modified document, `tss_verify` detects hash mismatch and rejects the proof.
3. **Tampering Experiment B (Manipulated Timestamp):** When the timestamp integer t inside `token.json` is altered by even 1 second, ECDSA signature verification fails immediately.
4. **Tampering Experiment C (Corrupted Signature):** When a single byte in the DER hex signature string is modified, OpenSSL's `EVP_DigestVerifyFinal` returns an invalid signature code.